

Introducción a los sistemas distribuidos

Sistema distribuido: definición y características

Un sistema distribuido es aquel en que los componentes localizados en computadores conectados en red comunican y coordinan sus acciones únicamente mediante el paso de mensajes. La motivación de los sistemas distribuidos es la de compartir recursos, entendiendo por recursos componentes de hardware o de software.

Los sistemas distribuidos presentan las siguientes características:

Concurrencia

Cada integrante de la red realiza trabajos independientes, pero también pueden compartir recursos en modo concurrente. Se debe proteger al sistema de los probables conflictos de concurrencia.

Inexistencia de reloj global

Cada máquina tiene su propio reloj y estos tienen límite de precisión, por lo tanto, a través de ellos no se puede lograr una sincronización perfecta.

Fallos independientes

Cualquier componente de la red puede fallar sin que el resto de los componentes perciban este fallo.

Ejemplos de sistemas distribuidos:

- Internet: es un sistema distribuido muy grande constituido por una colección de redes de diferentes tipos interconectadas y permite que un programa que se está ejecutando en cualquier parte dirija mensajes a otros programas en cualquier otra parte. Los programas se pueden ejecutar en programas conectados a la red e interactúan mediante el pasaje de mensajes empleando un medio de comunicación común.
- Intranets: es una porción de Internet, con las mismas características, pero tiene un límite de acción que permite hacer cumplir con políticas de seguridad. Puede estar

constituida por una o varias redes LAN en este último caso enlazadas por conexiones *backbone*. Una intranet puede brindarlos mismos servicios de Internet, pero limitados a su zona de acción y también podría integrarse a Internet a través de un *router*.

Sistema operativo distribuido: definición

Es aquel que administra los recursos de una colección de computadoras independientes que aparecen ante los usuarios del sistema como una única computadora.

Definición de términos

Resulta conveniente definir en este momento ciertos términos altamente vinculados con los sistemas operativos distribuidos:

Servicio

Es el encargado de gestionar en un sistema de computadoras una colección de recursos relacionados que presenta su funcionalidad a los usuarios y aplicaciones. Por ejemplo, un servicio de archivos brinda las operaciones que permiten leer, borrar y escribir archivos.

Servidor

Es el programa en ejecución (proceso) en un computador en red que acepta peticiones de los programas que se están ejecutando en otros computadores para realizar un servicio y responder adecuadamente.

Clientes

Son los programas en ejecución (procesos) solicitantes de servicios al servidor. Cuando un cliente realiza una petición se dice que realiza una invocación al servidor.

Ventajas y desventajas de los sistemas distribuidos

Ventajas

Economía

Los microprocesadores proporcionan mejor relación precio / rendimiento que cualquier *mainframe*.

Velocidad

Un sistema distribuido puede tener mayor poder de cómputo que un *mainframe*.

Distribución inherente

Ciertas aplicaciones son distribuidas en forma inherente, porque utilizan máquina que están separadas a cierta distancia. Por ejemplo, una cadena de supermercados tiene su stock, pero conviene que cada supermercado maneje el suyo, pero sin embargo en cierto momento podría interesar a cierto sector conocer el stock total de todas las sucursales.

Confiabilidad

Al estar distribuido el procesamiento en muchas máquinas, la caída de una de ellas sólo generaría la caída de un pequeño sector del sistema.

Crecimiento por incrementos

Se puede añadir poder de cómputos en pequeños incrementos.

Desventajas

Seguridad

Un acceso sencillo también se aplica a datos secretos.

Software

Aún existe poco software para los sistemas distribuidos.

La red se puede saturar o causar otros problemas.

Breve clasificación del hardware

La clasificación realizada por Michael Flynn (1966) es la más usada, aunque en el mejor caso sólo se la puede tomar como una sencilla aproximación. Esta clasificación de Flynn está basada en dos conceptos: corrientes o flujos de instrucciones y corrientes o flujos de datos. La primera corresponde a un contador de programa. Un sistema con n CPU's tiene n contadores, y por consiguiente n corrientes de instrucción. Una PC típica tiene un contador de programa y una corriente de instrucción. Un flujo de datos consiste de un conjunto de operandos. Un programa que calcula el promedio de una lista de temperaturas tiene un flujo de datos. Otro que calcula la temperatura promedio de cada uno de 100 termómetros esparcidos en todo el mundo tiene 100 flujos de datos.

Una computadora con un flujo de instrucciones y uno de datos se llama SISD (*Single Instruction Stream, Single Data Stream*). Todas las computadoras tradicionales de un procesador caen dentro de esta categoría desde las computadoras personales hasta los grandes mainframes. Las computadoras SISD representan la mayoría de las máquinas seriales. Tienen una UCP que ejecuta una instrucción en un momento dado y busca o guarda un dato en un momento dado. En la actualidad casi ninguna computadora es SISD pura porque la mayoría de los procesadores modernos tienen distintos grados de paralelismo.

La siguiente categoría es SIMD (*Single Instruction Stream, Multiple Data Stream*), con un flujo de instrucciones y varios flujos de datos. Este tipo se refiere a computadoras con un procesador y varias unidades aritmeticológicas para así ejecutar una misma instrucción con varios juegos de datos en forma simultánea o paralela.

Se puede dividir al grupo SIMD en dos subgrupos. El primero para supercomputadoras numéricas y otras máquinas que operan con vectores o matrices, realizando la misma operación con cada elemento del vector o matriz. En el segundo grupo están aquellas máquinas de tipo paralelo, en las que una unidad de control maestro envía instrucciones a muchas unidades aritmeticológicas independientes.

La categoría MISD (*Multiple Instruction Stream, Single Data Stream*), corresponde a un sistema con un flujo de varias instrucciones y un flujo de datos. En esta categorización hay

n procesadores cada uno recibiendo una instrucción diferente y operando sobre el mismo flujo de datos.

Por último, está el esquema MIMD (*Multiple Instruction Stream, Multiple Data Stream*), que significa un grupo de CPU's independientes, cada una con su propio contador de programa y datos, que operan como parte de un sistema más grande. Todos los sistemas distribuidos y la mayoría de los procesadores en paralelo caen dentro de esta categoría. Las MIMD son las más complejas, pero son también las que potencialmente ofrecen una mayor eficiencia en la ejecución concurrente o paralela. Aquí la concurrencia implica que no sólo hay varios procesadores operando simultáneamente, sino que además hay varios procesos ejecutándose también al mismo tiempo.

Se puede dividir a la categoría MIMD en dos grupos: uno formado por aquellas computadoras que tienen una memoria central compartida, que por lo general se llaman multiprocesadores, sistemas paralelos o sistemas centralizados y el otro por aquellas que no la comparten. Estas últimas han recibido diversos nombres como multicomputadoras, computadoras de memoria privada, computadoras distribuidas o computadoras de memoria desarticulada.

Cabe acotar que, la taxonomía de Flynn ha demostrado funcionar bastante bien para la tipificación de sistemas, y se ha venido usando desde hace varias décadas por la mayoría de los arquitectos de computadoras. Sin embargo, los avances en tecnología y diferentes topologías, han llevado a sistemas que no son tan fáciles de clasificar dentro de los 4 tipos de Flynn. Por ejemplo, los procesadores vectoriales no encajan adecuadamente en esta clasificación, ni tampoco las arquitecturas híbridas. Para solucionar esto se han propuesto otras clasificaciones, donde los tipos SIMD y MIMD de Flynn se suelen conservar, pero que sin duda no han tenido el éxito de la de Flynn.

Clasificación del software

Cada una de las estructuras de hardware que se acaban de analizar requerirá un tipo de sistema operativo. Sin embargo, a éstos no se los puede agrupar tan fácilmente como al hardware. Se podría hacer una división sencilla clasificando a los sistemas operativos en débilmente acoplados y fuertemente acoplados. Los primeros permiten que las máquinas y los usuarios de un sistema distribuido sean independientes entre sí, pero interactúen en cierto grado cuando sea necesario. Por ejemplo, un grupo de computadoras donde cada una tiene su procesador, su memoria, su propio disco y su propio sistema operativo, pero

comparten ciertos recursos: un plotter, una impresora láser, cualquier otro periférico o una base de datos en una red LAN. Por su parte, los fuertemente acoplados podrían ser el caso de un multiprocesador dedicado a la ejecución de un programa de ajedrez en paralelo, donde a cada procesador se le asigna un tablero para su evaluación y dedica su tiempo en la evaluación de este tablero y todos los que se puedan generar a partir de él. Al finalizar la evaluación el procesador informa el resultado y se le asigna un nuevo tablero.

Si se observa que se han desarrollado cuatro tipos de sistemas distribuidos según el hardware y dos según el software, entonces deberían existir ocho combinaciones de hardware y software; sin embargo, sólo existen tres, pues para el usuario es transparente el tipo de conexión, y son: el sistema operativo en red, el sistema operativo distribuido y el sistema operativo multiprocesador. La siguiente tabla es un resumen de las características de cada una de estas clases de sistema operativo.

Elemento	Sistema Operativo de Red	Sistema Operativo Distribuido	Sistema Operativo de Multiprocesador
¿Se ve como un monoprocesador virtual?	No	Sí	Sí
¿Todos tienen que ejecutar el mismo sistema operativo?	No	Sí	Sí
¿Cuántas copias del sistema operativo existen?	N	N	1
¿Cómo se logra la comunicación?	Archivos compartidos	Mensajes	Memoria compartida
¿Se requiere un acuerdo entre los protocolos de la red?	Sí	Sí	No
¿Existe una cola de ejecución?	No	No	Sí
¿Existe una semántica bien definida para los archivos compartidos?	Por lo general no	Sí	Sí

Modelos de sistema

Todo modelo de un sistema distribuido trata sobre el contenido de sus partes y las relaciones que existan entre ellas. Los más conocidos son el modelo cliente-servidor y el modelo de procesos “de igual a igual” (*peer to peer*). Los sistemas distribuidos, presentan ciertas dificultades que los diseñadores deben tener bien en cuenta, entre las cuales se destacan:

Modos de utilización muy variables

Los componentes de los sistemas están sujetos a grandes variaciones en la carga de trabajo.

Amplio rango de entorno

Normalmente deberán coexistir hardware, sistemas operativos y entornos de red diferentes.

Problemas internos

Relojes no sincronizados, actualizaciones de datos conflictivas, muchas formas de fallo en hardware y software, implicando a componentes individuales del sistema.

Amenazas externas

Están expuestas a ataques a la integridad, al secreto de los datos y a la denegación del servicio.

El término *arquitectura de software* se refiere a la estructuración del software como capas o módulos en función de los servicios ofrecidos y solicitados entre procesos localizados en el mismo o en diferentes computadores.

La plataforma está conformada por la conjunción del hardware y el sistema operativo. Se pueden mencionar como ejemplo diferentes plataformas como:

- Windows para Intel x86.
- Sun OS para Sun Sparc.
- Mac OS para Power PC.

- Linux para Intel x86.

Middleware es una capa de software que tiene como propósito enmascarar la heterogeneidad de las plataformas y proporcionar un modelo de programación conveniente para los programadores de aplicaciones. Proporciona bloques para construir componentes de software que puedan trabajar con otros sistemas distribuidos. Entre otras cosas mejora la comunicación entre programas de aplicación soportando:

- Procedimientos de invocación remota.
- Comunicación entre un grupo de procesos.
- Notificación de eventos.
- Replicación de datos compartidos.
- Transmisión de datos multimedia en tiempo real.

Los ejemplos más utilizados de middleware son paquetes de llamadas a procedimientos como RPC de SUN, ISIS o IV9000 de BULL. Entre los productos y estándares de middleware orientado a objetos están:

- CORBA (*Common Object Request Broker Architecture* De OMG).
- Invocación a objetos remotos en JAVA (RMI, *Remote Method Invocation*).
- Modelo común de objetos distribuidos de Microsoft (DCOM, *Distributed Common Objects Model*).
- Modelo de referencia para proceso distribuido abierto de la ISO/ITU-T (RM-ODP *Referente Model for Open Distributed Processing*).

Respecto de las limitaciones del middleware se puede decir que, aunque ha conseguido mucho en la simplificación de la programación en los sistemas distribuidos, todavía algunos aspectos de confiabilidad necesitan soporte del nivel de aplicación. Los programas distribuidos requieren comprobaciones, mecanismos de corrección de errores y medidas de seguridad, donde algunas de las cuales necesitan datos de direcciones de la capa de aplicación y no les basta con la de *middleware*.

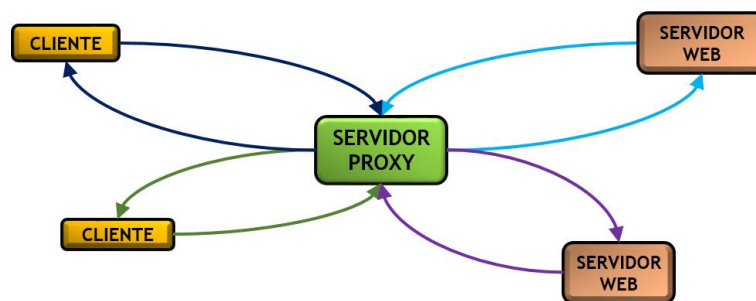
La arquitectura de un sistema es su estructura, en función de sus componentes especificados individualmente; la cual se diseñará con el objeto de asegurar que la misma satisfaga demandas actuales y futuras sobre el sistema. Para poder analizar los modelos arquitectónicos se pueden clasificar los procesos de sistemas distribuidos en:

- Servidores.
- Clientes e iguales.

Los modelos arquitectónicos que se detallarán pueden proporcionar sólo una versión simplificada de los patrones de distribución más importantes. Seguidamente se presentan los principales modelos arquitectónicos sobre los que se basa la división de responsabilidades de los componentes del sistema (aplicaciones, servidores y otros procesos).

Modelo cliente-servidor

Es la interacción de procesos clientes que invocan y procesos servidores que responden, en computadoras separadas y con el fin de acceder a recursos compartidos que los clientes gestionan. Los procesos servidores pueden ser a su vez clientes de otros servidores. La siguiente figura representa un ejemplo de modelo cliente-servidor:



¿Qué requisitos de diseño deberían presentar las arquitecturas distribuidas? En la actualidad compartir recursos en sistemas distribuidos no es inconveniente, pero aún hay un tema de cuidado que es el hecho de compartir datos en gran escala y en forma continua y efectiva.

Sistemas distribuidos sincrónicos

Un sistema distribuido sincrónico está definido por los siguientes límites:

- Tiempo de ejecución de cada etapa definido por su límite inferior y superior que debe ser suficientemente conocido.
- Cada mensaje transmitido debe tener un tiempo límite conocido.
- Cada proceso tiene un reloj local, cuya tasa de error o deriva debe ser menor a un límite conocido.

Sistemas distribuidos asincrónicos

Un sistema distribuido asincrónico es aquél sobre el cual no existen limitaciones de:

- Velocidad de procesamiento.
- Retardo de transmisión de mensajes.
- Tasas de error o derivas de relojes.

El modelo asincrónico es muy útil y aplicable en los sistemas distribuidos, un ejemplo de este modelo es Internet, aunque para ciertas aplicaciones como, por ejemplo, las multimediales, no es posible utilizar este modelo.

Descripción de algunos sistemas operativos distribuidos

A continuación, algunos detalles de distintos sistemas operativos distribuidos:

AMOEBA

Este sistema operativo se originó en Ámsterdam, Holanda en 1981, como proyecto de investigación en cómputo distribuido y paralelo. Fue diseñado por Andrew S. Tanenbaum y tres de sus estudiantes de doctorado, Frans Kaashoek, Sape J. Mullender y Robbert van Renesse.

El objetivo principal del proyecto AMOEBA era construir un sistema operativo distribuido y transparente. Un usuario común podría no percibir la diferencia entre AMOEBA y un sistema tradicional de tiempo compartido como UNIX, pero la diferencia real es que el que trabaja en AMOEBA entra, edita, mueve programas y archivos entre varias máquinas dispersas en la red y entre estas máquinas están los servidores de procesos, de archivos, de directorios, de cómputos, pero el usuario no es consciente de ello; porque todo parece como un sistema ordinario de tiempo compartido.

Una característica de AMOEBA es que cuando el usuario entra al sistema lo hace como un todo y no como una máquina específica, es decir que todos los recursos pertenecen al sistema como un todo y son controlados por él.

MACH

Mach nace de un sistema anterior llamado RIG (*Rochester Intelligent Gateway*) que se inició en la Universidad de Rochester en 1975. Fue escrito para una minicomputadora de 16 bits de Data General.

En 1986 después de varias evoluciones de RIG con un nuevo proyecto aparece MACH en su primera versión, para implementarlo en una VAX 11/784 que era un multiprocesador de 4 CPU, posteriormente se realizaron versiones para IBM PC/RT y para SUN 3. A pesar de tener posibilidad de su uso en redes, en esa época se concebía más como un sistema para un multiprocesador que para una red.

Los objetivos principales de MACH eran:

- Proporcionar una base para la construcción de otros sistemas operativos.
- Permitir acceso transparente a los recursos de red.
- Explotar el paralelismo tanto en el sistema como en las aplicaciones.
- Hacer que MACH se pueda transportar a una colección más grande de máquinas.

CHORUS

Nace en un instituto de investigación de Francia, INRIA, en 1980 como proyecto de investigación en sistemas distribuidos. Desde sus orígenes han aparecido 4 versiones numeradas como 0, 1 ,2 y 3. La idea inicial era modelar aplicaciones distribuidas como colección de procesos estructurados, cada uno de los cuales alternaban entre la realización de una operación atómica y la ejecución de un paso de comunicación. Cada máquina del sistema ejecutaba el mismo núcleo que controlaba a los procesos, la comunicación, los archivos y los dispositivos de E/S. Su primera versión fue desarrollada en Pascal interpretado y se ejecutó, en una colección de procesadores 8086 conectados mediante una red anillo, a partir de 1982.

La versión 1 (1982-1984) fue escrita para el multiprocesador SM90 que constaba de 8 procesadores 68020 de MOTOROLA en bus común, donde un procesador utilizaba UNIX y los otros siete ejecutaban CHORUS, utilizando el de UNIX para atender los servicios del sistema y las E/S. También atendía varias SM90 conectadas mediante ETHERNET. Esta versión también fue escrita en PASCAL, pero, en este caso, compilado y no interpretado.

La versión 2 (1984-1986) fue una rediseñada y escritura del sistema el lenguaje C y se diseñó de manera tal que las llamadas fuesen compatibles con UNIX en el nivel de código fuente y esto significaba que se podía recompilar los programas existentes en LINUX en CHORUS. En esta versión se agregó un soporte para las aplicaciones distribuidas incluyendo la ejecución remota y los protocolos para la asignación de nombres y la localización distribuida.

La versión 3 se inició en 1987 y marcó la transición de un sistema de investigación a uno de aplicación comercial, pues los desarrolladores salieron de la Universidad y formaron una empresa, CHORUS SYSTEMS, para continuar con los desarrollos y su comercialización. Desapareció en esta versión el concepto de procesos estructurados alternando con transacciones atómicas y se introdujo el concepto de llamada a procedimientos remotos (RPC) como el modelo comunicacional del sistema.

Para que esta versión pudiera ser una versión comercial se debió aumentar la capacidad de emulación de UNIX y se agregó la compatibilidad binaria con el mismo, de manera tal que se pudiesen correr programas en UNIX sin necesidad de recompilarlos.

Los objetivos del proyecto CHORUS fueron evolucionando con el tiempo de la misma manera que fue evolucionando su arquitectura. Los objetivos que en la actualidad mantienen vigencia son:

- Emulación de UNIX de alto rendimiento.
- Uso en sistemas distribuidos.
- Aplicaciones de tiempo real.
- Integración de la programación orientada a objetos en CHORUS.

Modos de comunicación entre sistemas distribuidos según el tipo de hardware

Según el tipo de hardware, es posible enumerar los siguientes modos de comunicación entre los sistemas distribuidos:

Tipo de hardware	Modo de comunicación
Fuertemente acoplado	Al tener memoria común se comunican a través de un buffer común.
Débilmente acoplado	Al no tener memoria compartida la comunicación se debe realizar a través de RPC (<i>Remote Procedure Control</i> – Llamadas a procedimientos remotos). Cuando un proceso A quiere comunicarse con otro B, primero A debe construir el mensaje en su propio espacio de direcciones de memoria, luego ejecuta una llamada al sistema para que el sistema operativo busque el mensaje y lo envía a través de la red hacia B.

Para poder realizar esa comunicación mediante mensajes se necesita acordar una serie de aspectos que ocupan una amplia gama de niveles (desde el más bajo de transmisión de los bits hasta los de más alto nivel) donde se explicita cómo debe expresarse la información, por ejemplo:

- Qué tensión hay que utilizar para enviar un bit 0 y cuál para enviar un bit 1.
- Cómo sabe el receptor cuál es el último bit del mensaje.

- Cómo detectar si un mensaje ha sido perdido o dañado.
- Cuál es la longitud de la cadena de datos y cómo se representan.

En una posible implementación del paso de mensajes los procesos hacen uso de los servicios de un módulo de paso de mensajes pudiéndose escribir las peticiones de servicio en términos de primitivas y parámetros. Aquí, una primitiva especificará la función que se quiere hacer mientras que los parámetros son usados para transferir los datos y la información de control.

La estructura de las primitivas variará según cual sea el software de paso de mensajes utilizado. Esta estructura puede implementarse como una llamada a un procedimiento o como un mensaje propiamente dicho a un proceso que sea parte del sistema operativo. Habitualmente la primitiva *Send* se utiliza cuando un proceso quiere enviar un mensaje, siendo sus parámetros el identificador del proceso receptor y el contenido del mensaje, y es el módulo de paso de mensajes el encargado de construir la unidad de datos que incluirá a estos elementos y que será enviada a la computadora que aloja al proceso receptor del mensaje utilizando algún tipo de servicios de comunicación, como el TCP/IP. Al recibir el equipo receptor la unidad de datos, el servicio de comunicación pasa esta al módulo de paso de mensajes que examinará el campo identificación de proceso y almacenará al mensaje en el búfer de proceso receptor. Ante esta situación, el proceso receptor debe explicitar su deseo de recibir un mensaje. Esto lo hace determinando un área de buffer e informando al módulo de paso de mensajes mediante la primitiva *Receive*. Otra opción sería que, al momento de recibir un mensaje el módulo de paso de mensajes identifique cuál es el proceso destinatario mediante una señal *Receive* para luego dejar disponible el mensaje recibido en un buffer compartido.

Otras características destacadas del paso de mensajes son:

El servicio puede ser confiable o no confiable

El servicio confiable es aquel que, si es posible, garantiza la entrega. Para poder cumplir con esto, el servicio realiza una comprobación de errores, acuse de recibo, retransmisión y reordenamiento de mensajes desordenados, de forma tal que no necesita informar al proceso emisor de que el mensaje ha sido enviado sin problemas. Por su parte, el servicio de paso de mensajes no confiable envía el mensaje por la red, pero no indica ni el éxito ni el fracaso del envío. Ante este hecho, los procesos que necesiten una confirmación de que

el mensaje ha sido entregado correctamente, deberán enviar y recibir mensajes para cumplir con esta necesidad.

El servicio puede ser bloqueante (sincrónico) o no bloqueante (asincrónico)

Las primitivas no bloqueantes no suspenden a un proceso como resultado de realizar un *Send* o *Receive*, de forma que, cuando un proceso realiza una primitiva *Send*, el sistema operativo devuelve el control al proceso no bien el mensaje fue puesto en la cola para transmitirlo o se lo ha copiado. Aunque estas primitivas proveen un buen uso de los servicios de paso de mensajes por parte de los procesos, tienen como desventaja la dificultad de chequear y depurar a los programas que utilicen estas primitivas.

En un servicio bloqueante, un *Send* no devolverá el control al proceso emisor hasta que el mensaje ha sido transmitido, si el servicio no es confiable, o hasta que el mensaje haya sido enviado y se haya recibido el acuse de recibo, cuando el servicio es confiable. En el caso de un *Receive* bloqueante, éste no devolverá el control hasta que el mensaje haya sido colocado en el búfer correspondiente.

Llamada a un Procedimiento Remoto (RPC)

El modelo cliente servidor es una forma apropiada de implementar un sistema operativo distribuido, pero el inconveniente es que los procedimientos de *Send* y *Receive* se basan en operaciones de E/S que no son las más apropiadas. En 1984 BIRELL y NELSON postularon una forma distinta de abordar el problema de implementar el sistema distribuido que consiste en que, si un proceso en la máquina "A" desea llamar a un procedimiento en la máquina "B", el proceso que realiza la llamada a "A" se suspende y se realiza la ejecución en "B". La información se transporta de un lado a otro por parámetros y regresa en el resultado del procedimiento. En este caso el programador no se preocupa de la transferencia de mensajes o de las operaciones de E/. Esta técnica se la denomina llamada a un procedimiento remoto o RPC. Por lo general el procedimiento que hace la llamada se llama cliente y el procedimiento que llamó se lo llama servidor.

En este esquema, lo que se quiere hacer es que una llamada a un procedimiento remoto sea lo más parecida posible a una llamada local. Luego, para llamar a un procedimiento remoto, el cliente debe enlazarse con una rutina de biblioteca conocida como resguardo del cliente, la cual representará al servidor en el espacio de direcciones del cliente. Asimismo, el servidor se enlazará con un procedimiento conocido como resguardo del

servidor. Ambos procedimientos ocultarán el hecho de que la llamada al procedimiento desde el cliente al servidor no es local.

Los pasos para llevar a cabo una RPC son los siguientes:

1. Ocurre cuando el cliente llama al resguardo del cliente. Esta llamada es una llamada a un procedimiento local, donde los parámetros se meten en la pila de la manera habitual.
2. Es cuando el resguardo del cliente empaqueta los parámetros, o *marshaling*, en un mensaje y realiza una llamada al sistema para enviarlo.
3. Sucede cuando el *kernel* envía el mensaje de la máquina cliente a la máquina servidor.
4. Cuando el *kernel* pasa el paquete entrante al resguardo del servidor (que por lo general llama a *receive* desde antes).
5. Cuando el resguardo del servidor llama al procedimiento del servidor. La respuesta sigue la misma ruta en dirección opuesta.

Lo importante en este desarrollo es que el procedimiento cliente sólo hace una llamada a procedimiento local al resguardo del cliente, el cual tiene el mismo nombre que el procedimiento del servidor. Puesto que el procedimiento del cliente y el resguardo del cliente pertenecen al mismo espacio de direcciones, los parámetros se pasan por valor o por referencia. Asimismo, un procedimiento llama al procedimiento del servidor en su espacio de direcciones, con los parámetros que espera. Así se lleva a cabo la comunicación remota al simular una llamada a un procedimiento local.

Sin embargo, este esquema presenta algunas dificultades, a saber:

- No es posible, o al menos puede resultar muy complicado, el uso de parámetros tipo puntero porque el cliente y el servidor se encuentran en distintos espacios de direcciones.
- Aunque el lenguaje permita el uso de parámetros tipo vector sin una longitud definida, le resultará imposible al cliente poder empaquetar los parámetros porque no tiene forma de determinar qué tan grandes son.
- No siempre se pueden deducir los tipos de los parámetros.
- El uso de las variables globales empleado tanto por los procedimientos que llaman como por los que son llamados o empleadas como parámetros. Si quien llama se traslada a una computadora remota, el código fallará porque las variables globales ya no están compartidas.

Grids (mallas)

Un *grid* es un conjunto inmenso de computadoras con las siguientes características:

- Las computadoras que lo forman se encuentran geográficamente dispersas.
- Esas máquinas son bastante heterogéneas.
- Se conecta mediante una red privada o Internet.
- Ofrece un conjunto de servicios para sus usuarios.
- Es una colección de computadoras independientes.
- Las computadoras que lo forman están, generalmente, en dominios administrativos distintos.
- Las computadoras ejecutan un nivel común de middleware para permitir que los programas y usuarios accedan a todos los recursos de una manera conveniente y consistente.
- Su principal objetivo es la compartición de los ciclos de la CPU como así también el hardware especializado y las bases de datos.

Usualmente un *grid* funciona de la siguiente forma:

1. Toda computadora integrante del *grid* ejecuta un grupo de programas que gestionan la computadora y la integran al *grid*. Habitualmente este software se encarga de la autenticación y el inicio de sesión de los usuarios remotos, de anunciar y descubrir recursos, programar y ubicar trabajos, etcétera.
2. Siempre que un usuario realiza un trabajo, el software del *grid* primero determina en dónde se tienen recursos de hardware, software y de datos libres para la ejecución del trabajo, luego envía el trabajo a ese lugar, prepara su ejecución y más tarde devuelve los resultados obtenidos al usuario.

Como ejemplo de middleware para un *grid* se tiene a Globus Toolkit utilizable en varias plataformas y que puede soportar varios esquemas de *grid*. Globus suministra un entorno de trabajo que les permite a los usuarios compartir computadoras, archivos y muchos otros recursos de forma flexible y con mucha seguridad sin tener que sacrificar la autonomía local. Debido a estas cualidades es que se lo usa como plataforma en la construcción de una gran cantidad aplicaciones distribuidas.

Bibliografía consultada

- Carretero Pérez, J., De Miguel Anasagasti, P., García Carballeira, F. y Pérez Costoya, F. (2001). *Sistemas Operativos: Una visión aplicada* (2ª ed.). Buenos Aires: Mc Graw Hill.
- Silberschatz, A. y Galvin, P. B. (2006). *Fundamentos de Sistemas Operativos* (7ª ed.). México: Addison Wesley Logman.
- Stallings, W. (2005). *Sistemas operativos: Principios de diseño e interioridades* (5ª ed.). Madrid: Pearson/Prentice Hall.
- Tanenbaum, A. S. y Van Steen, M. (2009). *Sistemas operativos modernos* (3ª ed.). México: Pearson/Prentice Hall.